PHISHEYE

Feature Summary by Frontend and Backend Role

SIH-1454 · Multi-channel scam intelligence · React dashboard + FastAPI engine + Chrome extension

This note maps every implemented PHISHEYE capability to the layer that owns it. The React dashboard and the Chrome extension are presentation clients. They collect input, show risk, and never compute a phishing score themselves. FastAPI is the single detection brain. The same URL sent from the website or from the extension returns the same score.

Honest limits: this build is a multi-signal rule engine plus live threat lists, not a trained CNN or LLM. Risk 0–100 is fused evidence, not a calibrated probability of fraud. Community reports are a signal, not proof.

1. Who does what

Layer	What it is	What it owns	What it must not do
Frontend (React + Vite)	Web dashboard at http://127.0.0.1:5173	Pages, forms, charts, Simple/Technical view, login UI, Safe Open warning, export display	Does not score URLs, decode QR, query PhishTank, or store the database
Backend (FastAPI + SQLite)	API at http://127.0.0.1:8000	Auth, URL/message/QR analysis, threat intel, psychology, risk fusion, history, reports, stats	Does not render the dashboard or inject the Chrome overlay
Chrome extension (MV3, v1.8)	Unpacked browser client	Intercept navigation, show the gate and popup, send URL + page title/heading, badge, Analyze All Links	Does not detect phishing locally. If the API is down it says unavailable, never SAFE

2. Feature map (frontend vs backend)

Each row is one user-facing feature. Frontend = what the operator sees or types. Backend = the API and engine that make it true.

Feature	Frontend role	Backend role
User signup / login	Login and Signup pages; store JWT in localStorage; protect dashboard routes	POST /api/v1/user/signup and /login; bcrypt password hash; JWT; GET /profile
Landing / marketing page	Public landing copy, dark/light theme, navigation to login	None (static React page)
URL scan	Scanner URL tab and Dashboard quick scan; show score, level, reasons, Simple + Technical view	POST /api/v1/threat/check-url and /analyze/url; URLChecker + decision engine
Pasted message scan (SMS / WhatsApp / text)	Message and WhatsApp/SMS tabs; user pastes content; show psychology findings and extracted links	POST /api/v1/analyze/content and /threat/check-message; extract URLs; psychology 2.0 (English + Hinglish)
Email text scan	Email tab with From, display name, Reply-To fields	POST /api/v1/analyze/email; sender mismatch analysis plus message + URL pipeline
QR code scan	QR tab: upload an image	POST /api/v1/analyze/qr; OpenCV decode; then the same content analyzer on extracted URLs
Screenshot scan	Screenshot tab: upload image, optional pasted URL hint	POST /api/v1/analyze/screenshot; QR always; OCR only if Tesseract is installed
Short-link unwrapping	Show original URL, final URL, and redirect hops in the result panel	Expand shorteners; SSRF block for localhost, private IPs, and cloud metadata
Domain existence and age	Show “does not exist” or “N days old” in reasons	DNS / WHOIS-style age lookup; extra risk for very new domains
Brand impersonation	Show “looks like a fake PayPal / SBI / …” in Simple View	Typos, leetspeak (paypa1), lookalike letters, free-host phishing names
Psychological threat language	Show urgency / fear / greed findings from pasted text	Psychology service: English + Hinglish phrases; cannot alone force HIGH/CRITICAL
Threat intelligence	Show feed match text (PhishTank, OpenPhish, URLhaus, Phishing Army, Google Safe Browsing)	Public dumps + live PhishTank checkurl for suspected reports; GSB if API key is set; miss is not SAFE
Cloudflare / browser phishing interstitial	Extension sends page title and heading after load; dashboard does not scrape pages	If title/heading says “Suspected Phishing” (and the site is not trusted), raise HIGH/CRITICAL
Risk score 0–100 and levels	Risk badge, meters, Simple View (plain English) and Technical View (tags, evidence)	Heuristic score + intel floor; LOW <30, MEDIUM 30–49, HIGH 50–69, CRITICAL ≥70

Feature	Frontend role	Backend role
Simple vs Technical view	Toggle and dual panels on scan result	build_simple_view / build_technical_view; never claims “completely safe” or “100%”
Safe Open	Warn before opening HIGH/CRITICAL links in a new tab; does not rewrite the original message	Returns classification so the client knows whether to block the click
Multi-link panel	List every extracted link with its own score; Analyze All Links in the extension	Extract http(s) URLs from text/HTML; score each through URLChecker
Community report (Scam / Risky / Safe)	Flag buttons on scan result (signed-in users)	POST /api/v1/report; rate-limited; stored as reputation signal, not proof
Detection feedback	Helpful yes/no after a scan	POST /api/v1/feedback; stored for later review of false positives
Scan history	History page lists recent scans; open or delete own rows	SQLite content_scans / url_checks; GET /scans; DELETE /scans/{id} (owner only); raw message text is not stored
Dashboard statistics	Stat cards, pie/bar charts (Recharts): overview, risk mix, threat types, daily summary, recent URLs	GET /api/v1/stats/* computed from SQLite
Reports page	Show community report summary and the operator’s own flags	GET /api/v1/stats/reports-summary and /reports/mine
Export report	Download / display a sanitized report from a scan	Export builder omits raw message body and secrets
Settings (web)	Theme, local notify/strict toggles, show profile, logout	GET /api/v1/user/profile only; preferences stay in the browser
Privacy page	Explain what is and is not collected	Privacy model in API responses (e.g. scans do not store raw pasted SMS)
Security advisor	Static FAQ for non-technical users (HIGH vs CRITICAL, fake login tips)	None — rule-based copy in React, not an LLM chatbot
Chrome real-time protection	Extension popup, gate, overlay, badge OK/MED/HIGH/CRIT, settings, login	Same POST /api/v1/threat/check-url (plus page_signals). Extension never invents a local score
Scan-first navigation gate	Extension holds the click, shows the gate, auto-opens LOW, asks on MEDIUM, keeps HIGH/CRITICAL closed	Returns risk_level / safe so the extension can decide
Health / intel status	Dashboard error if API is down	GET /health; GET /api/v1/threat/intel-status (which feeds and API keys are loaded)

3. Frontend pages and their backend calls

React page / UI	Operator action	Backend endpoints used
Landing /	Read product pitch, go to login/signup	None
Login / Signup	Create account or sign in	POST /api/v1/user/signup, /login
Dashboard /dashboard	See totals, charts, recent URLs, quick URL/screenshot scan	GET /stats/overview, /daily-summary, /threat/recent-urls, /stats/risk-distribution, /stats/threat-types; scan APIs
Scanner /scanner	Choose URL, Message, Email, WhatsApp/SMS, Screenshot, or QR	POST /analyze/content, /analyze/url, /analyze/email, /analyze/qr, /analyze/screenshot, /threat/check-url
Scan result /scan-result	Read Simple + Technical view, flag, Safe Open	Report/feedback APIs after a scan payload already returned
History /history	Browse past scans, delete own analysis	GET /api/v1/scans, DELETE /scans/{id}
Reports /reports	Community flags and summary	GET /stats/reports-summary, /reports/mine, POST /report
Statistics /statistics	Deeper charts: sources, timeline, risk mix	GET /stats/sources, /timeline, /risk-distribution, /threat-types
Privacy /privacy	Read data-handling notes	None (copy)
Settings /settings	Theme and local preferences; show profile	GET /api/v1/user/profile
Advisor /advisor	Read “what should I do” guidance	None
Extension popup / gate / settings	Live tab scan, Analyze All Links, account, backend URL	POST /threat/check-url, /analyze/content, /user/login, GET /user/profile

4. Backend engine (not visible as its own screen)

These modules run only on the server. The frontend just displays their output.

Backend module	Job	How the frontend uses it
url_checker.py	Lexical heuristics, TLS notes, ads/clutter, free-host patterns, interstitial text	Reasons list and risk_score
threat_intel.py + feed_cache.py	PhishTank (dump + live suspected reports), OpenPhish, URLhaus, Phishing Army, Google Safe Browsing	Confirmed by … / unverified report lines
decision_engine.py	Intel cannot be averaged away by a low heuristic score	classification CONFIRMED_MALICIOUS / SUSPICIOUS / NO_MALICIOUS_EVIDENCE
risk_engine.py	Weighted fusion for multi-channel content (URL, psychology, brand, community)	scam_risk and model_confidence on message scans
psychology.py	Urgency, fear, greed, credential pressure; Hinglish phrases	Message-tab findings
brand_detector.py / sender_analyzer.py	Fake brand + sender mismatch	Email tab and Simple View
qr_analyzer.py	OpenCV QR decode from uploaded bytes	QR and Screenshot tabs
redirects.py + ssrf.py	Unwrap short links without fetching private networks	Redirect chain on the result card
ssl_certs.py	TLS certificate inspection	Technical view HTTPS notes
reputation.py	Community labels as a weak extra signal	Reports page and fused score
warnings.py	Simple English vs technical evidence packets	Dual view on every scan
auth.py + SQLite	Users, tokens, url_checks, content_scans, reports	Login, history, dashboard counts

5. Chrome extension 1.8 (frontend client of the same backend)

The extension is not a second detector. It is another frontend. Version 1.8 adds scan-first gating, PhishTank suspected-report handling on the server, and detection of Cloudflare-style warning pages via title/heading.

Extension UI	Frontend role	Backend role
Toolbar popup	Show host, headline, score, reasons, Scan page, Analyze all links, dashboard/settings links	POST /api/v1/threat/check-url { url, view: both, page_signals }
Scan-first gate / panel	Hold navigation, auto-open LOW, Open anyway on MEDIUM, keep HIGH/CRITICAL closed	Same check-url result (risk_level, safe)
On-page overlay / badge	Toast + OK/MED/HIGH/CRIT badge without opening the popup	Same scan payload
Content script	Collect button/iframe counts, page title, heading, short snippet; outline risky links	page_signals used for clutter score and vendor-interstitial detection
Extension settings	Backend URL, dashboard URL, auto-scan, intercept, warnings, login	Login uses POST /api/v1/user/login; scans use the configured API base

6. What this app cannot do (do not claim in a demo)

Claim to avoid	Reality
Reads native phone SMS from Chrome	Blocked. User must paste the message. Android companion is planned.
Opens private Gmail / WhatsApp inboxes	Only pasted text, or visible links already rendered on the page
Trained deep-learning phishing CNN / LLM advisor	Rule engine + threat lists. Advisor page is static FAQ.
Calibrated “true probability of fraud”	Score is fused evidence. Model confidence is signal agreement, not accuracy.
Community flag proves a site is phishing	One extra signal, rate-limited, not legal proof
PostgreSQL in production today	Live database is SQLite
Screenshot OCR always works	QR always; OCR only if Tesseract is installed on the machine

7. One-line demo script

Frontend: open the dashboard, paste https://paypa1.com or a PhishTank suspected URL, show Simple View. Backend: the same FastAPI check-url call is what the extension uses. Extension: reload PHISHEYE 1.8, visit a listed phishing URL — the gate should block it, and a Cloudflare “Suspected Phishing” page must not be labelled NO THREAT.

Stack: React 19 + Vite + Tailwind (frontend) · FastAPI + SQLite + OpenCV (backend) · Manifest V3 extension. Optional keys: GOOGLE_SAFE_BROWSING_API_KEY, PHISHTANK_API_KEY, URLHAUS_AUTH_KEY.

PHISHEYE · Feature roles document · Generated from the SIH-1454 codebase